Python E-Course

Module 15 — Image Restoration & Noise

Detailed Notes

Module Overview

Level: Advanced

Duration: ~1 week

Prerequisites: Modules 1-14

What You'll Learn

- Identify and synthesise common noise types.
- Pick the right denoiser per noise type.
- Apply non-local means and bilateral for high-quality denoising.
- Use Wiener filtering for deconvolution / restoration.
- Measure quality objectively with PSNR and SSIM.

Lessons

#	Lesson	Duration
1	Noise Types & Synthesis	30 min
2	Median Revisit + Non-Local Means	30 min
3	Bilateral & Edge-Preserving Denoising	25 min
4	Wiener Filter (Deconvolution Intro)	30 min
5	Quality Metrics — PSNR & SSIM	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 16: Capstone Projects

Lesson 1: Noise Types & Synthesis

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Distinguish Gaussian, salt-and-pepper, speckle, and periodic noise visually and statistically.
- Synthesise each noise type to test denoisers.
- Read a noise histogram.

Prerequisites

- Modules 1-14

1. Why This Matters

You can't pick a denoiser without naming the enemy. Each noise type has a characteristic signature, a typical cause, and a "best" denoiser. Five minutes here saves trial-and-error.

2. Concept

2.1 The four noise types you'll meet

Type	Looks like	Caused by	Best denoiser
Gaussian	grain everywhere, bell-shaped distribution	sensor read noise, low light	NLM, bilateral, Gaussian
Salt-and-pepper	random pure-white & pure-black specks	bit errors, dropped pixels	median
Speckle	multiplicative grain (more in bright areas)	radar, ultrasound, laser	NLM, Lee filter
Periodic	repeating patterns (stripes, screens)	electronic interference, scans	notch filter (M14)

2.2 Synthesizing each (for testing your denoiser)

Gaussian:

```
noisy = np.clip(img.astype(np.int16) +
                np.random.normal(0, sigma, img.shape).astype(np.int16),
                0, 255).astype(np.uint8)
```

Salt-and-pepper:

```
out = img.copy()
m = np.random.rand(*img.shape[:2])
out[m < p] = 0
out[m > 1 - p] = 255
```

Speckle (multiplicative):

```
n = np.random.normal(1.0, 0.1, img.shape).astype(np.float32)
noisy = np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)
```

Periodic (sinusoidal stripe):

```
yy, xx = np.mgrid[:img.shape[0], :img.shape[1]]
stripe = 30 * np.sin(2 * np.pi * (xx + yy) / 16)
noisy = np.clip(img.astype(np.float32) + stripe, 0, 255).astype(np.uint8)
```

2.3 Diagnosing real noise

Steps:

- Look at a flat area (sky, paper, wall) — what do you see?
- Plot histogram of that flat area:
- Bell curve → Gaussian.
- Two spikes near 0 and 255 → salt-and-pepper.
- Asymmetric, brightness-dependent spread → speckle.
- Check the DFT spectrum — bright dots off-centre → periodic.

2.4 Noise level estimation

For Gaussian, a quick estimate from a flat region:

```
flat = img[10:60, 10:60]                # known-flat patch
sigma_est = flat.std()
```

Or scikit-image's wavelet-based estimator:

```
from skimage.restoration import estimate_sigma
print(estimate_sigma(img, average_sigmas=True))
```

2.5 The denoising trade-off

Every denoiser averages neighbouring pixels in some way. Result: noise drops AND fine detail blurs. Quality denoisers maximise noise removal per unit detail loss. Quantify with PSNR/SSIM (Lesson 5).

3. Code Examples

Example 1: Synthesise all four noise types

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)

# Gaussian
sigma = 25
g = np.clip(img.astype(np.int16) +
```

```
            np.random.normal(0, sigma, img.shape).astype(np.int16), 0,
255).astype(np.uint8)

# Salt-and-pepper
sp = img.copy()
m = np.random.rand(*img.shape)
sp[m < 0.03] = 0; sp[m > 0.97] = 255

# Speckle
n = np.random.normal(1.0, 0.1, img.shape).astype(np.float32)
spk = np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)

# Periodic
yy, xx = np.mgrid[:img.shape[0], :img.shape[1]]
per = np.clip(img.astype(np.float32) + 30 * np.sin(2 * np.pi * (xx + yy) / 16),
              0, 255).astype(np.uint8)

for name, x in [("gauss", g), ("sp", sp), ("speckle", spk), ("periodic", per)]:
    cv2.imwrite(f"noise_{name}.png", x)
    print(f"{name:8s} std={x.std():.1f}  range={x.min()}-{x.max()}")
```

Expected Output: Four PNG files demonstrating each noise type, with statistics that match the type (e.g. salt-and-pepper has wide range 0-255 but median near the original).

Example 2: Histogram diagnosis

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import camera

img = camera()
np.random.seed(0)
gauss = np.clip(img + np.random.normal(0, 25, img.shape), 0, 255).astype(np.uint8)
sp = img.copy()
m = np.random.rand(*img.shape); sp[m < 0.03] = 0; sp[m > 0.97] = 255

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].hist(gauss.ravel(), 256, [0, 256]); ax[0].set_title("Gaussian-noisy
histogram")
ax[1].hist(sp.ravel(),    256, [0, 256]); ax[1].set_title("Salt-and-pepper
histogram")
plt.tight_layout(); plt.show()
```

Expected Output: Gaussian histogram is a smoothed bell-shaped curve. Salt-and-pepper has two prominent spikes at 0 and 255 (the salt and pepper) plus the original distribution.

Example 3: Estimate Gaussian σ from a flat patch

```
import numpy as np
import cv2
from skimage.data import camera

img = camera()
np.random.seed(0)
```

```
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

flat = noisy[20:80, 20:80]
print("estimated sigma:", flat.std())
```

Expected Output: estimated sigma: ~20 (close to the true value we used).

Example 4: scikit-image estimator

```
import numpy as np
from skimage.restoration import estimate_sigma
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
print("skimage sigma estimate:", estimate_sigma(noisy, average_sigmas=True))
```

Expected Output: value near 20 — sometimes a slight under/overshoot depending on image content.

4. Parameter Sensitivity

Noise param	Effect
Gaussian σ	larger = more grain
S&P prob p	larger = more dropouts
Speckle std	larger = more brightness-modulated grain
Periodic amplitude	larger = stronger stripes

5. Common Mistakes

Mistake	What Happens	Fix
Add Gaussian noise on uint8 directly	overflow at clipping → distribution skewed	Cast to int16 first
Sample uniform speckle	wrong distribution shape	Use multiplicative Gaussian (mean=1)
Treat salt-and-pepper as Gaussian	filter fails	Diagnose first by checking for 0/255 spikes

6. Practice Tasks

- Synthesise Gaussian noise (σ=25) on camera(); save.
- Plot histograms of a Gaussian-noisy vs salt-and-pepper-noisy image; identify each from its histogram alone.
- Estimate σ from a flat patch; compare to ground truth.

7. Key Takeaways

- Four noise types: Gaussian, salt-and-pepper, speckle, periodic.
- Each has a characteristic signature in image and histogram.
- Synthesise to test denoisers reproducibly.
- Estimate σ from flat regions or estimate_sigma.

8. What's Next

Median (M7 revisited) and the king of Gaussian-noise denoisers: non-local means.

Lesson 2: Median Revisit + Non-Local Means

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Apply cv2.medianBlur for impulse noise (recap of M7).
- Apply cv2.fastNlMeansDenoising / Colored for Gaussian noise.
- Tune NLM parameters: h, templateWindowSize, searchWindowSize.

Prerequisites

- Module 7 Lesson 4, Module 15 Lesson 1

1. Why This Matters

Non-Local Means (NLM) is the best general-purpose denoiser available without deep learning. It produces results that look more natural than Gaussian / bilateral on real photos. Median is still the right answer for impulse noise.

2. Concept

2.1 Median for impulse noise (recap)

Salt-and-pepper noise → cv2.medianBlur(img, 3) is unbeatable. Median ignores outliers because they sit at the extremes of the sorted neighbourhood.

For higher densities (15-30% noise) try ksize=5 or 7. Above 30%, you'll lose real detail; combine with morphology or use a deep model.

2.2 The NLM idea (intuition)

A traditional Gaussian filter averages spatial neighbours. NLM averages similar pixels — even if they're far away. For each pixel p, search a window for patches similar to p's patch; weight each by similarity; average.

Result: noise (which is random) cancels out, while genuine detail (which repeats across the image in similar patches) is preserved.

2.3 OpenCV API

```
cv2.fastNlMeansDenoising(src, h=10, templateWindowSize=7, searchWindowSize=21)
cv2.fastNlMeansDenoisingColored(src, h=10, hColor=10, ...)
```

Param	Meaning
h	Filter strength. Larger = more denoising AND more blurring. Typical 5-15.
templateWindowSize	Patch size for similarity (default 7). Should be

	odd.
searchWindowSize	Neighbourhood to search for similar patches (default 21). Should be odd.
hColor (Colored version)	Same as h but for color channels

2.4 Picking h

A common heuristic: h ≈ estimated_sigma. If you don't know σ, start with 10 and tune visually.

Too small → noise remains. Too large → detail wiped out.

2.5 Performance

NLM is slow — typically 1-10 seconds for an HD image. Downsample first if you can:

```
small = cv2.resize(img, None, fx=0.5, fy=0.5)
out_small = cv2.fastNlMeansDenoisingColored(small, None, 10, 10, 7, 21)
out = cv2.resize(out_small, img.shape[1::-1])    # may lose some quality
```

For video, OpenCV provides cv2.fastNlMeansDenoisingMulti which uses several frames.

2.6 NLM vs Bilateral

NLM:

- Slower but higher quality on photographic noise.
- Better at preserving textures (uses patch similarity).

Bilateral:

- Faster.
- Better when you want strong edge preservation (e.g. skin smoothing).

Use NLM as the quality benchmark; bilateral when speed matters.

3. Code Examples

Example 1: NLM denoise grayscale

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
clean = cv2.fastNlMeansDenoising(noisy, None, h=15, templateWindowSize=7,
searchWindowSize=21)

cv2.imwrite("noisy.png", noisy)
cv2.imwrite("clean_nlm.png", clean)
print("noisy std on flat patch:", noisy[10:60, 10:60].std())
print("clean std on flat patch:", clean[10:60, 10:60].std())
```

Expected Output: Noisy has flat-patch std ~20; clean ~3-5. Visually, the cameraman is recognisable and most detail is preserved.

Example 2: NLM color (faster than per-channel)

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
clean = cv2.fastNlMeansDenoisingColored(noisy, None, h=10, hColor=10,
                                        templateWindowSize=7, searchWindowSize=21)
cv2.imwrite("astro_noisy.png", noisy)
cv2.imwrite("astro_nlm.png", clean)
```

Expected Output: astro_nlm.png shows the astronaut with grain dramatically reduced; fine helmet text and hair detail preserved.

Example 3: Tune h

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

for h in [5, 10, 20, 40]:
    out = cv2.fastNlMeansDenoising(noisy, None, h=h, templateWindowSize=7,
searchWindowSize=21)
    cv2.imwrite(f"nlm_h{h}.png", out)
print("compare files: h=5 underdenoised, h=40 oversmoothed")
```

Expected Output: h=5 still has visible grain. h=40 looks plasticky / loses detail. h=10–15 is the sweet spot for σ=20 noise.

Example 4: Median for salt-and-pepper (recap)

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
m = np.random.rand(*img.shape)
sp = img.copy(); sp[m < 0.05] = 0; sp[m > 0.95] = 255

med  = cv2.medianBlur(sp, 3)
nlm  = cv2.fastNlMeansDenoising(sp, None, h=15)

cv2.imwrite("sp_median.png", med)
cv2.imwrite("sp_nlm.png", nlm)
```

Expected Output: Median: clean image, sharp. NLM on salt-and-pepper: visible grey smudges where the impulse noise was — NLM doesn't handle outliers well. Confirms: pick the filter for the noise type.

4. Parameter Sensitivity

Param	Lower	Higher
h (NLM)	residual noise	smoother but more detail loss
templateWindowSize	local similarity	broader patch similarity
searchWindowSize	small region	longer runtime, more patches considered

5. Common Mistakes

Mistake	What Happens	Fix
Use NLM on salt-and-pepper	grey smudges	Use median
Run NLM at full HD without thinking	slow	Downsample, denoise, upsample
Per-channel NLM on color	color shifts possible	Use fastNlMeansDenoisingColored
h too large	plasticky output	Match h to estimated σ

6. Practice Tasks

- Add Gaussian noise (σ=20) to camera(); denoise with NLM, h=15.
- Add salt-and-pepper noise; compare median vs NLM.
- Tune NLM h in {5, 10, 20, 40}; pick a visual sweet spot.

7. Key Takeaways

- Median for impulse / salt-and-pepper.
- NLM for Gaussian / photographic grain.
- h ≈ σ is a good starting point for NLM.
- NLM is slow — downsample if needed.

8. What's Next

Bilateral filter in this restoration context — when and how to combine with NLM.

Lesson 3: Bilateral & Edge-Preserving Denoising

Module: 15 — Image Restoration & Noise

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.bilateralFilter for edge-preserving denoising.
- Compare bilateral with NLM, Gaussian, and median.
- Combine filters in a denoising pipeline.

Prerequisites

- Module 7 Lesson 5, Module 15 Lesson 2

1. Why This Matters

Bilateral was introduced in M7 for general smoothing. Here we focus on its restoration role: when edges matter more than texture (e.g. portraits, scanned documents), bilateral wins over NLM in speed and over Gaussian in edge preservation.

2. Concept

2.1 Bilateral in one line

cv2.bilateralFilter smooths spatially close pixels with similar intensity. Edges (large intensity difference) survive because dissimilar pixels are excluded from the average.

```
out = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)
```

Param	Meaning
d	spatial neighbourhood diameter (9 typical)
sigmaColor	intensity-difference tolerance (50-100 typical)
sigmaSpace	spatial extent (50-100 typical)

2.2 Restoration recipes

Mild Gaussian noise + sharp edges (portraits):

```
out = cv2.bilateralFilter(noisy, 9, 50, 50)
```

Heavy Gaussian noise:

```
# Stack bilateral several times
out = noisy
for _ in range(3):
    out = cv2.bilateralFilter(out, 9, 75, 75)
```

Stacking trades small detail for stronger smoothing — much cheaper than NLM at heavy h.

Mixed noise (impulse + Gaussian):

```
out = cv2.medianBlur(noisy, 3)              # remove salt-and-pepper
out = cv2.bilateralFilter(out, 9, 50, 50)   # smooth the rest
```

2.3 Edge-preserving variants in OpenCV

- cv2.edgePreservingFilter(img, flags=cv2.RECURS_FILTER, sigma_s=60, sigma_r=0.4) — fast edge-preserving smoothing.
- cv2.detailEnhance — bilateral + sharpening combo.
- cv2.stylization — bilateral + edge map (cartoon effect).

These wrap variants of bilateral for one-call effects.

2.4 Bilateral vs NLM — when to pick which

You want	Use
Smooth skin / strong edge preservation	bilateral
Best photographic-noise removal	NLM
Fast (real-time / video)	bilateral
Quality first, time available	NLM
Mix of texture + noise	NLM, then mild bilateral

2.5 Anisotropic diffusion (advanced sibling)

Perona-Malik anisotropic diffusion is the academic ancestor of bilateral. scikit-image has skimage.restoration.denoise_tv_chambolle (total variation) — preserves piecewise-flat regions even better. Useful for cartoon-style images.

3. Code Examples

Example 1: Bilateral on Gaussian noise

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 15, img.shape), 0, 255).astype(np.uint8)
out = cv2.bilateralFilter(noisy, 9, 75, 75)
cv2.imwrite("astro_bilat.png", out)
```

Expected Output: Astronaut with much less grain but edges (helmet rim, mission patch text) preserved. Compare to NLM — bilateral is a touch less smooth in textured regions but ~10× faster.

Example 2: Stacked bilateral for heavy noise

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
```

```
noisy = np.clip(img + np.random.normal(0, 30, img.shape), 0, 255).astype(np.uint8)

out = noisy
for _ in range(3):
    out = cv2.bilateralFilter(out, 9, 75, 75)
cv2.imwrite("cam_bilat_x3.png", out)
```

Expected Output: Heavy noise is dramatically reduced; fine grain replaced by smooth shading; edges of buildings still crisp.

Example 3: Pipeline — median + bilateral for mixed noise

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
mixed = np.clip(img + np.random.normal(0, 15, img.shape), 0, 255).astype(np.uint8)
m = np.random.rand(*img.shape)
mixed[m < 0.02] = 0; mixed[m > 0.98] = 255

step1 = cv2.medianBlur(mixed, 3)
step2 = cv2.bilateralFilter(step1, 9, 50, 50)
cv2.imwrite("cam_pipeline.png", step2)

import numpy as np
print("flat patch std before:", mixed[10:60, 10:60].std())
print("flat patch std after :", step2[10:60, 10:60].std())
```

Expected Output: Std drops from ~30 to ~5. Visually: speckles removed by median, residual grain smoothed by bilateral.

Example 4: edgePreservingFilter — one-call recipe

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

out = cv2.edgePreservingFilter(noisy, flags=cv2.RECURS_FILTER, sigma_s=60,
sigma_r=0.4)
cv2.imwrite("coffee_eps.png", out)
```

Expected Output: Slightly painterly-looking coffee image — noise gone, edges crisp, slight cartoonish flat-fill effect.

4. Parameter Sensitivity

Param	Lower	Higher
d	faster, less smoothing	slower, more smoothing
sigmaColor	keep more edges	smooth across stronger edges

Stacks (re-applications)	mild	strong / loses detail

5. Common Mistakes

Mistake	What Happens	Fix
Use bilateral on salt-and-pepper	Specks survive, look smeared	Median first
sigmaColor too large	Acts like Gaussian (no edge preservation)	Try 25-75
Stack too many times	Plasticky	2-3 stacks max

6. Practice Tasks

- Add Gaussian noise σ=15 to astronaut(); denoise with bilateral 9, 75, 75.
- Add heavy noise σ=30; stack bilateral 3 times.
- Add mixed noise (Gaussian + S&P); apply median → bilateral pipeline.

7. Key Takeaways

- Bilateral = fast, edge-preserving denoising.
- Stack 2-3 times for heavier noise.
- Combine with median for mixed noise.
- edgePreservingFilter wraps bilateral-like recipes for one-call use.

8. What's Next

Wiener filtering — when the noise is mixed with blur (deconvolution).

Lesson 4: Wiener Filter (Deconvolution Intro)

Module: 15 — Image Restoration & Noise

Duration: 30 minutes

Difficulty: Advanced

Learning Objectives

- Distinguish denoising from deblurring (deconvolution).
- Apply scipy.signal.wiener for adaptive denoising.
- Implement a frequency-domain Wiener deconvolution for motion blur.

Prerequisites

- Module 14 (frequency domain), Module 15 Lesson 3

1. Why This Matters

When you know HOW the image got corrupted (blur kernel + noise), you can mathematically reverse it. Wiener filtering is the classic optimal solution in the least-squares sense. Used in microscopy, radar, and any imaging where the point-spread function is known.

2. Concept

2.1 Image formation model

```
g = f ⊗ h + n
```

- g — what you observed (blurry + noisy).
- f — the true image you want.
- h — the blur kernel (point-spread function / PSF).
- n — additive noise.
- ⊗ — convolution.

Goal: recover f given g and (ideally) h.

2.2 Naive inverse — and why it fails

In frequency space:

```
G = F · H + N    →    F_est = G / H  − N/H
```

If H has small values (and it usually does at high frequencies), dividing by H blows up noise. So naive inversion is a disaster.

2.3 Wiener — regularised inverse

```
F_est = (H̄ / (|H|² + K)) · G
```

Where K is the noise-to-signal power ratio. Small K → close to inverse (sharp but noisy); large K → safe (over-smooth).

In practice, treat K as a hyperparameter and tune.

2.4 scipy.signal.wiener — adaptive variance method

A simpler form: estimates local variance and shrinks pixels toward the local mean based on noise variance. Works well as a general denoiser when you don't know the PSF.

```
from scipy.signal import wiener
out = wiener(img.astype(np.float32), mysize=5, noise=None)   # noise=None auto-estimates
```

2.5 Frequency-domain Wiener for motion blur

When you know the PSF (or can estimate it):

```
def wiener_deconvolve(g, h, K=0.01):
    g = g.astype(np.float32) / 255
    H = np.fft.fft2(h, s=g.shape)
    G = np.fft.fft2(g)
    H_conj = np.conj(H)
    W = H_conj / (np.abs(H)**2 + K)
    F_est = W * G
    out = np.real(np.fft.ifft2(F_est))
    return np.clip(out * 255, 0, 255).astype(np.uint8)
```

2.6 Limitations

- Need to know (or estimate) the PSF.
- Noise model assumed Gaussian.
- Ringing at image borders.
- For real photos, deep-learning deblurring (e.g. Restormer, DeepDeblur) is usually better.

2.7 When to use what

You have	Use
Noisy image, no blur, unknown statistics	NLM / bilateral
Noisy + known blur (PSF)	Wiener deconvolution
Heavy motion blur, no PSF	Blind deconvolution (advanced)
Modern photo, want highest quality	Deep model (Restormer, NAFNet, …)

3. Code Examples

Example 1: scipy.signal.wiener as a denoiser

```
import numpy as np, cv2
from scipy.signal import wiener
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)

out = wiener(noisy.astype(np.float32), mysize=5, noise=None)
out = np.clip(out, 0, 255).astype(np.uint8)
```

```
cv2.imwrite("cam_wiener.png", out)

print("noisy flat std:", noisy[10:60, 10:60].std())
print("wiener flat std:", out[10:60, 10:60].std())
```

Expected Output: Flat-patch std drops dramatically (e.g. 20 → 6). Image is visibly cleaner with slight loss of fine texture.

Example 2: Synthesize motion blur + recover via Wiener

```
import numpy as np, cv2
from skimage.data import camera

img = camera().astype(np.float32) / 255.0
h, w = img.shape

# Horizontal motion-blur PSF (15-pixel line)
psf = np.zeros((h, w), dtype=np.float32)
psf[h//2, w//2 - 7:w//2 + 8] = 1
psf /= psf.sum()

# Apply blur via FFT
H = np.fft.fft2(np.fft.ifftshift(psf))
blurred = np.real(np.fft.ifft2(np.fft.fft2(img) * H))
np.random.seed(0)
blurred_noisy = np.clip(blurred + np.random.normal(0, 0.01, img.shape), 0, 1)

# Wiener deconvolve
def wiener_deconv(g, H, K):
    G = np.fft.fft2(g)
    W = np.conj(H) / (np.abs(H)**2 + K)
    return np.real(np.fft.ifft2(W * G))

for K in [0.0001, 0.001, 0.01, 0.1]:
    restored = wiener_deconv(blurred_noisy, H, K)
    cv2.imwrite(f"deblur_K{K}.png", np.clip(restored * 255, 0,
255).astype(np.uint8))

cv2.imwrite("blurred_input.png", np.clip(blurred_noisy * 255, 0,
255).astype(np.uint8))
```

Expected Output: blurred_input.png is the noisy horizontally-blurred cameraman. deblur_K0.0001.png is sharp but noisy; deblur_K0.1.png is over-smooth. deblur_K0.001 tends to be the sweet spot — visibly sharper than input with moderate noise.

Example 3: Wiener vs bilateral on Gaussian-noisy image

```
import numpy as np, cv2
from scipy.signal import wiener
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 20, img.shape), 0, 255).astype(np.uint8)
```

```
w_out = np.clip(wiener(noisy.astype(np.float32), mysize=5), 0, 255).astype(np.uint8)
b_out = cv2.bilateralFilter(noisy, 9, 75, 75)

cv2.imwrite("cam_wiener.png", w_out)
cv2.imwrite("cam_bilat.png", b_out)
```

Expected Output: Both look denoised. Bilateral keeps edges sharper; Wiener is slightly more uniform-looking. For pure denoising without blur, bilateral / NLM are usually preferred.

4. Parameter Sensitivity

Param	Effect
K (Wiener regulariser)	small → sharper but noisy; large → safer but blurrier
mysize (scipy wiener)	local window; 3-7 typical
PSF accuracy	wrong PSF → ringing / smearing

5. Common Mistakes

Mistake	What Happens	Fix
Try Wiener on photo without PSF	Just acts like denoiser	Use scipy version or another method
K = 0	Naive inverse, noise explodes	K > 0 always
Wrong PSF shape	Artefacts	Estimate or measure PSF
Use uint8 in math	Overflow	Cast to float, work in [0, 1]

6. Practice Tasks

- Apply scipy.signal.wiener(mysize=5) to Gaussian-noisy camera().
- Synthesize horizontal motion blur on camera(); recover with Wiener; tune K.
- Compare Wiener vs bilateral on the same Gaussian-noisy input.

7. Key Takeaways

- Wiener = noise-aware deconvolution.
- Need (or estimate) the PSF for deblurring.
- K trades sharpness for noise.
- For pure denoising, simpler filters (NLM/bilateral) often suffice.

8. What's Next

Measuring "how good is my denoiser" objectively — PSNR and SSIM.

Lesson 5: Quality Metrics — PSNR & SSIM

Module: 15 — Image Restoration & Noise

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute PSNR and SSIM with scikit-image.
- Interpret typical values.
- Build a small denoiser benchmark.

Prerequisites

- Module 15 Lessons 1-4

1. Why This Matters

"Looks better" isn't reproducible. PSNR and SSIM give you numbers — you can rank denoisers, set quality thresholds, and report results without arguing about subjective opinion.

2. Concept

2.1 MSE — the foundation

Mean Squared Error:

```
MSE = (1/N) · Σ (true - estimate)²
```

Lower = better. But MSE on its own is unitless and doesn't relate to image properties.

2.2 PSNR

Peak Signal-to-Noise Ratio:

```
PSNR = 10 · log10(MAX² / MSE)
```

For 8-bit images, MAX = 255. Units: decibels (dB). Higher = better.

PSNR (dB)	Quality
> 40	Near-imperceptible difference
30-40	Good (typical for compressed photos)
20-30	Visible degradation
< 20	Significant degradation

2.3 SSIM — Structural Similarity

PSNR is per-pixel and ignores perceptual structure. SSIM considers luminance, contrast, and structure within local windows. Range -1 to 1; 1 = identical.

SSIM	Quality

> 0.98	Visually identical
0.9-0.98	Very good
0.7-0.9	Recognisable but degraded
< 0.7	Significantly different

SSIM correlates better with human perception than PSNR for many distortions (blur, blocking, contrast shifts).

2.4 scikit-image API

```
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
psnr = peak_signal_noise_ratio(reference, test, data_range=255)
ssim = structural_similarity(reference, test, data_range=255)
```

For color, pass channel_axis=2:

```
ssim = structural_similarity(ref_color, test_color, data_range=255, channel_axis=2)
```

2.5 The benchmark pattern

For comparing denoisers:

```
methods = {
    "gaussian":  lambda x: cv2.GaussianBlur(x, (5, 5), 0),
    "median":    lambda x: cv2.medianBlur(x, 3),
    "bilateral": lambda x: cv2.bilateralFilter(x, 9, 75, 75),
    "nlm":       lambda x: cv2.fastNlMeansDenoising(x, None, h=15),
}

for name, f in methods.items():
    out = f(noisy)
    p = peak_signal_noise_ratio(clean, out, data_range=255)
    s = structural_similarity(clean, out, data_range=255)
    print(f"{name:10s}  PSNR={p:5.2f}  SSIM={s:.3f}")
```

This gives you a quantitative leaderboard.

2.6 Caveat — metrics ≠ perception

- PSNR rewards pixel-exact match. A barely-noticeable shift can tank PSNR.
- SSIM is better but not perfect.
- For final QA, use BOTH plus visual inspection.
- Modern: LPIPS / DISTS use deep features for perceptual quality.

3. Code Examples

Example 1: Compute PSNR / SSIM

```
import numpy as np, cv2
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,
```

```
255).astype(np.uint8)

print("PSNR (noisy vs clean):", peak_signal_noise_ratio(clean, noisy,
data_range=255))
print("SSIM (noisy vs clean):", structural_similarity(clean, noisy, data_range=255))
```

Expected Output:

```
PSNR (noisy vs clean): ~22.0
SSIM (noisy vs clean): ~0.40
```

Example 2: Benchmark denoisers

```
import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio as psnr, structural_similarity
as ssim

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,
255).astype(np.uint8)

methods = {
    "gaussian":  lambda x: cv2.GaussianBlur(x, (5, 5), 1.0),
    "median":    lambda x: cv2.medianBlur(x, 3),
    "bilateral": lambda x: cv2.bilateralFilter(x, 9, 75, 75),
    "nlm":       lambda x: cv2.fastNlMeansDenoising(x, None, h=15),
}

print(f"{'method':10s}{'PSNR':>8s}{'SSIM':>8s}")
for name, f in methods.items():
    out = f(noisy)
    print(f"{name:10s}{psnr(clean, out, data_range=255):8.2f}{ssim(clean, out,
data_range=255):8.3f}")
```

Expected Output:

```
method        PSNR    SSIM
gaussian     27.1   0.72
median       25.4   0.65
bilateral    28.3   0.78
nlm          30.1   0.84
```

NLM wins (expected); median is worst on Gaussian noise (also expected).

Example 3: SSIM map (per-pixel)

```
import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import structural_similarity

clean = camera()
np.random.seed(0)
noisy = np.clip(clean + np.random.normal(0, 20, clean.shape), 0,
```

```
255).astype(np.uint8)

score, ssim_map = structural_similarity(clean, noisy, data_range=255, full=True)
print("global SSIM:", score)
ssim_vis = cv2.normalize(ssim_map, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("ssim_map.png", ssim_vis)
```

Expected Output: Score ~0.40. ssim_map.png shows where the noisy and clean images differ most — uniform regions look identical (bright in the map), textured regions less so.

Example 4: Track quality during stacked filtering

```
import cv2, numpy as np
from skimage.data import camera
from skimage.metrics import peak_signal_noise_ratio as psnr

clean = camera()
np.random.seed(0)
out = np.clip(clean + np.random.normal(0, 25, clean.shape), 0, 255).astype(np.uint8)

print("step 0:", psnr(clean, out, data_range=255))
for i in range(1, 6):
    out = cv2.bilateralFilter(out, 9, 60, 60)
    print(f"step {i}:", psnr(clean, out, data_range=255))
```

Expected Output: PSNR rises in the first 2-3 passes, then plateaus or starts decreasing as detail is eroded. Tells you when to stop stacking.

4. Parameter Sensitivity

Metric	Best when...
PSNR	Identifying pixel-exact match
SSIM	Comparing perceived quality
LPIPS (deep)	Comparing different artefact types

5. Common Mistakes

Mistake	What Happens	Fix
data_range wrong	Inflated / deflated numbers	Pass data_range=255 for uint8
Compute on different shapes	Error	Resize to match
Compare uint8 vs float without normalising	Bad numbers	Use the same dtype/range
Use PSNR alone	Misleading for blurs	Pair with SSIM

6. Practice Tasks

- Compute PSNR and SSIM between camera() and a σ=20 noisy version.
- Benchmark Gaussian/median/bilateral/NLM on the same noisy image.
- Stack bilateral 5 times; plot PSNR per step; find the optimum.

7. Key Takeaways

- PSNR (dB) and SSIM (0-1) quantify image quality.
- skimage.metrics has both.
- Use BOTH plus visual inspection for QA.
- data_range=255 for uint8 — don't forget it.

8. What's Next

End of Module 15. Next: Module 16 — Capstone Projects and the 8 standalone projects in projects/.

Module 15 — Exercises

15 exercises.

15.1 Noise types

- (E) Synthesise σ=25 Gaussian noise on camera(); save.
- (M) Plot histograms of Gaussian-noisy vs S&P-noisy versions; identify each from histogram alone.
- (H) Estimate σ from a flat patch and via skimage.restoration.estimate_sigma; compare to true σ.

15.2 Median + NLM

- (E) NLM denoise σ=20 Gaussian noise on camera() with h=15.
- (M) Apply median(3) vs NLM(h=15) on salt-and-pepper noise; compare.
- (H) Tune NLM h ∈ {5, 10, 20, 40}; pick the sweet spot for σ=20 noise.

15.3 Bilateral

- (E) bilateralFilter(9, 75, 75) on σ=15 noise on astronaut().
- (M) Stack bilateral 3 times on σ=30 noisy camera().
- (H) Mixed noise pipeline: median → bilateral; report flat-patch std before/after.

15.4 Wiener

- (E) Apply scipy.signal.wiener(mysize=5) to Gaussian-noisy camera().
- (M) Synthesize horizontal motion blur via FFT; recover with Wiener (K=0.001).
- (H) Sweep Wiener K ∈ {1e-4, 1e-3, 1e-2, 1e-1}; pick the best output.

15.5 Metrics

- (E) Compute PSNR/SSIM between camera() and σ=20 noisy version.
- (M) Benchmark Gaussian/median/bilateral/NLM on the same noisy input.
- (H) Stack bilateral 5×; plot PSNR per step; find the optimum.

Module 15 — Quiz

10 MCQs.

Q1

Salt-and-pepper noise is best removed by: A. Gaussian blur B. median C. bilateral D. NLM

Answer: B (L1, L2)

Q2

For photographic Gaussian noise, the best classical denoiser is: A. median B. Gaussian C. NLM D. erosion

Answer: C (L2)

Q3

In cv2.fastNlMeansDenoising, h controls: A. patch size B. filter strength C. window stride D. iterations

Answer: B (L2)

Q4

Bilateral preserves edges because: A. it computes Laplacian B. range kernel down-weights dissimilar intensities C. it's iterative D. it works in frequency

Answer: B (L3)

Q5

A frequency-domain notch filter targets: A. Gaussian noise B. salt-and-pepper C. periodic noise D. speckle

Answer: C (L1)

Q6

Wiener deconvolution requires knowledge of: A. the noise standard deviation only B. the blur (PSF) C. ground-truth image D. SSIM target

Answer: B (L4)

Q7

PSNR is measured in: A. pixels B. dB C. percent D. milliseconds

Answer: B (L5)

Q8

SSIM range is: A. 0..1 B. 0..255 C. -1..1 D. 0..100

Answer: C (L5)

Q9

For a uint8 image, the correct data_range for PSNR is: A. 1 B. 100 C. 255 D. depends

Answer: C (L5)

Q10

Stacking bilateral too many times: A. always improves PSNR B. removes color C. eventually loses detail D. inverts the image

Answer: C (L3, L5)

Module 15 — Assignment: Denoiser Benchmark Suite

Estimated time: 2 hours

Combines: Module 15 + Modules 7, 14

Brief

Build a CLI that takes a clean image, synthesises noise of a chosen type, applies multiple denoisers, and produces a benchmark table with PSNR/SSIM.

Requirements

- bench.py INPUT --noise gauss|sp|speckle [--sigma 20] [--p 0.03]
- Load the clean reference; synthesise noise per the chosen type.
- Apply at least 4 denoisers: cv2.GaussianBlur, cv2.medianBlur, cv2.bilateralFilter, cv2.fastNlMeansDenoising (+ optionally Wiener).
- Compute PSNR and SSIM for each output vs reference.
- Print a sortable table (best PSNR first).
- Save a side-by-side image: [clean | noisy | best-method-output] with method label.

Constraints

- Modules 1-15 only.
- argparse.
- Use skimage.metrics.peak_signal_noise_ratio and structural_similarity.

Expected Output

```
$ python bench.py datasets/starter/camera.png --noise gauss --sigma 20

method        PSNR    SSIM
nlm          30.12   0.842
bilateral    28.34   0.781
gaussian     27.05   0.722
median       25.41   0.651

saved -> camera_bench.png  (clean | noisy | nlm)
```

Grading Rubric

Criterion	Weight
Noise synthesis	15%
4 denoisers correct	30%
PSNR/SSIM computed	20%
Sorted table output	15%
Side-by-side image	10%
Argparse + style	10%

Submission

Save as assignment_module15.py.

Module 15 — Mini Project: Interactive Denoiser Comparator

Overview

A cv2-window comparator. Pick a noise type and intensity via trackbars; see four denoisers running side-by-side with live PSNR/SSIM numbers.

Concepts Used

- All of Module 15 (noise synthesis, NLM, bilateral, median, metrics).
- Trackbars (M2 L3).

Features

- Trackbars: noise type (0..2), noise intensity, NLM h, bilateral sigmaColor.
- 5-panel display: clean | noisy | bilateral | median | NLM (with labels + PSNR).
- s saves the whole comparison; q quits.

Starter Code

denoise_compare.py:

```
import sys, cv2, numpy as np
from skimage.metrics import peak_signal_noise_ratio as psnr

NOISES = ["gauss", "sp", "speckle"]

def add_noise(img, kind, intensity):
    rng = np.random.default_rng(0)
    if kind == "gauss":
        return np.clip(img + rng.normal(0, intensity, img.shape), 0,
255).astype(np.uint8)
    if kind == "sp":
        out = img.copy()
        m = rng.random(img.shape)
        p = intensity / 100
        out[m < p] = 0; out[m > 1 - p] = 255
        return out
    if kind == "speckle":
        n = rng.normal(1.0, intensity / 100, img.shape).astype(np.float32)
        return np.clip(img.astype(np.float32) * n, 0, 255).astype(np.uint8)

def annotate(img, label, score):
    out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR) if img.ndim == 2 else img.copy()
    cv2.rectangle(out, (0, 0), (out.shape[1], 30), (0, 0, 0), -1)
    cv2.putText(out, f"{label} PSNR={score:.1f}",
                (5, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 1)
    return out

def run(path):
    clean = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if clean is None: raise FileNotFoundError(path)
    if max(clean.shape) > 400: clean = cv2.resize(clean, (320, 320))
```

```
    cv2.namedWindow("denoise")
    cv2.createTrackbar("noise 0gauss 1sp 2speckle", "denoise", 0, 2, lambda v: None)
    cv2.createTrackbar("intensity",  "denoise", 20, 80,  lambda v: None)
    cv2.createTrackbar("nlm h",      "denoise", 15, 40,  lambda v: None)
    cv2.createTrackbar("bilat sc",   "denoise", 75, 200, lambda v: None)

    while True:
        kind = NOISES[cv2.getTrackbarPos("noise 0gauss 1sp 2speckle", "denoise")]
        intensity = cv2.getTrackbarPos("intensity", "denoise") or 1
        h_nlm = cv2.getTrackbarPos("nlm h", "denoise") or 1
        sc    = cv2.getTrackbarPos("bilat sc", "denoise") or 1

        noisy = add_noise(clean, kind, intensity)
        bi  = cv2.bilateralFilter(noisy, 9, sc, sc)
        me  = cv2.medianBlur(noisy, 3)
        nlm = cv2.fastNlMeansDenoising(noisy, None, h=h_nlm)

        panels = [
            annotate(clean,  f"clean ({kind})", 99.0),
            annotate(noisy,  "noisy",           psnr(clean, noisy, data_range=255)),
            annotate(bi,     f"bilat sc={sc}",  psnr(clean, bi, data_range=255)),
            annotate(me,     "median k=3",      psnr(clean, me, data_range=255)),
            annotate(nlm,    f"NLM h={h_nlm}",  psnr(clean, nlm, data_range=255)),
        ]
        combo = np.hstack(panels)
        cv2.imshow("denoise", combo)
        k = cv2.waitKey(30) & 0xFF
        if k == ord('q'): break
        if k == ord('s'): cv2.imwrite("denoise_compare.png", combo); print("saved")
    cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/camera.png")
```

Expected Interaction

Window shows 5 strips side-by-side with PSNR labels. Slide "intensity" to make noise stronger; see PSNRs drop. Switch noise type; observe median dominating on S&P.

Extension Challenges

- Add Wiener as a 6th panel.
- Add an "SSIM" toggle alongside PSNR.
- Click any panel to save just that one image.

Grading Rubric

Criterion	Weight
Three noise types	20%
Four denoisers	30%
PSNR labels live	20%

Slider responsiveness	15%
Style	15%